

RLoPlanner: Combining Learning and Motion Planner for UAV Safe Navigation in Cluttered Unknown Environments

Yuntao Xue^{1b} and Weisheng Chen^{1b}

Abstract—In this paper, we propose a hierarchical navigation framework named RLoPlanner that combines deep reinforcement learning algorithms and local motion planners, allowing unmanned aerial vehicles (UAVs) to perform navigation tasks safely and energy-efficiently in complex and unknown environments. This technique is critical to improve the performance of UAVs in environments without prior maps. Specifically, the framework high-level generates a stochastic policy through a deep reinforcement learning algorithm based on maximum entropy, which generates a local target based on raw sensor information. Then the low-level motion planner tracks the local goal to generate a smooth trajectory to the final target. Compared with existing end-to-end navigation methods, the proposed navigation framework generates trajectories that are smoother, more energy-efficient and more dynamically feasible. The framework also overcomes the drawback that the mapping and planning methods tend to fall into local minima. Our experiments in a simulated environment with random obstacles demonstrate that RLoPlanner outperforms state-of-the-art methods in terms of navigation success rate and kinematically compliant trajectories.

Index Terms—Deep reinforcement learning, maximum entropy DRL, motion planning, hierarchical navigation.

I. INTRODUCTION

UNMANNED aerial vehicles (UAVs) have gained increasing importance in applications such as logistics distribution, disaster relief, and military reconnaissance due to their long cruising time, low cost, and good concealment [1], [2], [3]. To perform these applications efficiently, autonomous navigation of UAVs is crucial, allowing them to reach the target from the starting point without collision. However, real-world environments full of obstacles pose significant challenges to autonomous navigation. For instance, dense urban buildings and complex forest branches reduce the possibility of safe UAV navigation. Given the fast flight speed and air movement of UAVs, even small collisions and errors can result in serious pedestrian injury and property damage. Accordingly, the recent research emphasis is

on developing a navigation algorithm enabling UAVs to safely and efficiently operate without prior maps.

Numerous comprehensive studies have examined the issue of autonomous navigation for UAV safety, with both traditional non-learning-based approaches and learning-based approaches being proposed. The non-learning-based methods can be classified into two categories. The first category involves methods that respond directly to raw sensor information and do not require the creation of an environmental map. This approach, referred to as perception and avoidance [4], [5], [6], is capable of quickly adapting to changes in the environment. For instance, in [4], a binocular camera was used to detect obstacles from a disparity map and a wall-following algorithm was employed for avoidance. In [5], dynamic and static objects were differentiated using temporal flow information obtained from bio-inspired temporal cameras, and control signals were generated using fast strategies to avoid approaching obstacles. All these methods can avoid obstacles without knowledge of the environment. However, as shown in [6], reactive obstacle avoidance methods perform poorly when obstacles are dense and have poor robustness to perceptual errors.

The second category of non-learning methods is based on simultaneous localization and mapping (SLAM), which involves the robot determining its own position while it moves and constructing a map of the environment incrementally [7], [8]. Subsequently, the robot performs path planning based on the established map to reach its navigation goal. This motion planning encompasses both path finding and trajectory optimization. A method for joint polynomial trajectory optimization in unconstrained quadratic programs is presented in [9]. This method enables the determination of the shortest collision-free path on the occupied map of the given environment. Helen et al. [10] introduced a continuous-time trajectory optimization approach that combines objective functions, such as the collision cost from CHOMP [11], to generate real-time safe obstacle avoidance trajectories. However, this approach is prone to getting stuck in local minima. B-splines with convex hull properties were utilized to represent trajectories in [12], and kinodynamic constraints were considered to further improve performance. Despite their ability to avoid obstacles and replanning in real-time, these methods overlook the issue of getting stuck in local minima. These two classes of non-learning navigation methods are dependent on motion planners to generate feasible and

Manuscript received 29 June 2023; revised 14 September 2023; accepted 2 November 2023. Date of publication 20 November 2023; date of current version 22 April 2024. This work was supported by the National Natural Science Foundation of China under Grants 62073254 and 92271101. The review of this article was coordinated by Dr. Chen Lv. (Corresponding author: Weisheng Chen.)

The authors are with the School of Aerospace Science and Technology, Xidian University, Xi'an 710071, China (e-mail: ytxue@stu.xidian.edu.cn; wshchen@126.com).

Digital Object Identifier 10.1109/TVT.2023.3330703

high-quality trajectories in a fraction of the time [13]. However, as shown in [14], the motion planner does not solve the problem of long-range planning beyond the field of view well in an unknown environment, when the UAV is prone to missing the planning of long-term goals and getting into a dilemma.

With the development of deep reinforcement learning in recent years, many researchers have developed learning-based UAV navigation methods [15], [16], [17] to address the shortcomings of non-learning-based methods. A technique for point navigation tasks in discrete environments was trained in [15] using a combination of PID and Q-learning algorithm, followed by validation on a physical UAV system. However, the discrete state space assumption of this algorithm restricts its applicability in practical scenarios, and the resulting trajectory is also discontinuous. To address these issues, Wang et al. [16] introduced the fast recurrent deterministic policy gradient algorithm (Fast-RDPG) for UAV navigation in large-scale unknown environments, which incorporates prior knowledge from the field of UAV navigation. Despite its effectiveness, the trajectory generated by this purely learning-based approach may not always be dynamically feasible and may lack smoothness. Additionally, the training process of the algorithm is hindered by sparse rewards and a lack of rewards for movements in free space. Even when rewards and penalties are designed based on prior knowledge of the navigation field, the algorithm requires a slow convergence speed and a significant amount of time for the RL agent to learn basic collision-free movements [18]. There are also works [19], [20] that combine end-to-end approaches with roadmaps for long-range navigation. For example, in [19], a short-range navigation policy was trained using RL, followed by a sampling-based path planner to provide a robot roadmap, but the approach was challenging in unknown environments.

As mentioned in the work [21], in unknown complex environments, deep reinforcement learning algorithms are proficient at deciding the appropriate exploration points after summarizing the environment structure, while local motion planners can quickly compute safe and energy-efficient trajectories to reach the target. In other words, DRL is adept at solving the problem of where to go, while MP is good at solving the problem of how to get there. Therefore, we propose the framework named RLoPlanner, a hierarchical UAV navigation framework that leverages the advantages of both by skillfully combining deep reinforcement learning and a **local planner**. The RLoPlanner adopts a model-free policy gradient algorithm to train a policy that generates local targets from raw observations. These local targets then guide the UAV to reach the global target through a collision-free path in unknown environments. The local targets are fed into the low-level motion planner, which generates smooth, dynamically feasible control sequences for execution on the UAV.

In the high-level deep reinforcement learning algorithm, we work to address the problems of sample complexity and slow convergence of the model-free based approach applied to UAV navigation. As previously demonstrated in [22], the maximum entropy policy [23] can enhance exploration ability and robustness through diverse action learning, but its Q-learning-based approach is not suited for continuous action spaces. In [24],

a continuous control task is modeled as a Markov Decision Process (MDP) and a soft policy gradient, derived through entropy regularization of expected reward targets, is proposed as a solution. To enable the UAV to have more chances of reaching the final target, we adopt the maximum entropy policy gradient algorithm with stochasticity and exploratory capability to generate local targets. Given the limited information available to the UAV through onboard sensors, the decision-making process for navigation constitutes a partially observable Markov Decision Process (POMDP). A model-free RL algorithm using recurrent neural networks is one method for dealing with partial observability. For Atari games with screen flicker (simulating partial observability), a deep recurrent Q network using a long short-term memory networks (LSTM) in place of the fully connected layer in [25] successfully incorporated temporal information and outperformed DQN. Using recurrent neural networks trained by temporal backpropagation, Heess et al. [26] enhanced the policy gradient technique and verified difficult memory problems on numerous partially observed physical control problems. To address the challenge of UAVs with limited field of view in partially observable domains, we add LSTM networks to the RL algorithm to enable UAVs to improve their decision making ability with the help of the entire historical trajectory learning experience. In summary, the selection of local target points in an unknown environment by the UAV is modeled as a POMDP and an actor-critic based recurrent maximum entropy policy gradient algorithm is designed to solve it.

For the low-level motion planner, inspired by Ego-planner, we use a compact representation of the environment to reduce the computational effort. Obstacle information is stored only when the trajectory encounters a new obstacle, and the main contribution of our work lies in the addition of a local target cost function to the penalty function. This allows for slight adjustments to the target points, thereby leading to the discovery of better trajectories. The entire RLoPlanner framework can be seen as modeled as a lifted POMDP, with the action space of the high-level RL algorithm is the sequence of local targets of the motion planner. This skillful combination is the key to solving the problems of slow RL convergence and the tendency of the motion planner to get trapped.

In summary, the main contributions of this paper are:

- 1) Aiming at the problem of autonomous navigation of UAV in unknown complex environment, a navigation framework combining recursive maximum entropy policy gradient algorithm and motion planner is proposed. Compared with end-to-end methods, the framework produces smooth and dynamic trajectories. Compared with the traditional re-planning algorithm, it can make the UAV overcome the local minimum and actively explore the environment.
- 2) The local target point selection problem in UAV navigation is modeled as POMDP, and a recursive maximum entropy policy gradient algorithm is derived to solve it.
- 3) A lightweight gradient-based trajectory planning algorithm is extended, and a local objective cost function is fused to generate a better trajectory.

The remainder of the paper follows. Section II introduces the background knowledge of POMDP and the reinforcement

learning algorithm based on maximum entropy. Section III builds the model for UAV navigation. The hierarchical UAV navigation framework and recurrent soft policy gradient algorithm are introduced in Section IV. Section V discusses the simulation results. Section VI summarizes this paper and discusses future work.

II. BACKGROUND

In this section, we first briefly introduce the POMDP model and then summarize the maximum entropy reinforcement learning algorithm, which is used as the upper-level local target generation policy for the proposed framework.

A. POMDP

In this work, we formulate the UAV navigation task as a partially observable Markov decision process with a continuous action space. The POMDP is defined by the seven-tuple $(S, A, T, R, \Omega, O, \gamma)$, where S represents the state set, A represents the action set, T represents the state transition probability, Ω is the observation set, O is the conditional observation transition probability, and $\gamma \in [0, 1)$ is the discount factor. At time step t , the state of the environment is $s_t \in S$, the agent selects an action $a_t \in A$, and the environment state transitions to the next state s_{t+1} based on $T(s_{t+1} | s_t, a_t)$. The agent receives a new observation $O(o_t | s_{t+1}, a_t)$ and a reward r_t at the same time. However, the agent does not have direct access to the state s_t and can only infer it from the observation set Ω based on the conditional observation transition probability $p(o_t | s_t)$. Reinforcement learning requires access to the entire history $h_t = (o_1, a_1, o_2, a_2, \dots, a_{t-1}, o_t)$ to make informed decisions. The goal of the agent is to learn a policy $\pi(h_t)$ that maximizes the expected discounted reward over the entire trajectory, defined as $E_\tau[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$, where the trajectory $\tau = (s_1, o_1, a_1, s_2, \dots)$ is drawn from the policy π distribution $p(s_1)p(o_1 | s_1)\pi(a_1 | h_1)p(s_2 | s_1, a_1)p(o_2 | s_2) \dots$.

In MDP, the action-value function Q^π can be defined as the expected future discounted reward for executing action a_t in state s_t and then following policy π . When the agent cannot access the state s , the action-value function Q^π can be defined according to the trajectory h_t as:

$$Q^\pi(h_t, a_t) = \mathbb{E}_{s_t | h_t} [r(s_t, a_t)] + \mathbb{E}_{\tau_{>t} | h_t, a_t} \left[\sum_{i=1}^{\infty} \gamma^i r(s_{t+i}, a_{t+i}) \right] \quad (1)$$

where $\tau_{>t} = (s_{t+1}, o_{t+1}, a_{t+1}, \dots)$ is the future trajectory.

B. Maximum Entropy Reinforcement Learning

Different from standard reinforcement learning, the goal of maximum entropy RL is to maximize an entropy-regularized objective, defined as increasing the expected sum of rewards with a discounted entropy term. The benefit of the entropy term is to increase the randomness and exploration capabilities

of the policy.

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}^\pi(\cdot | s_t)] \quad (2)$$

where α is a parameter that determines the relative importance of the entropy item to the reward, $\mathcal{H}^\pi(\cdot | s_t) = -\sum_{a_t} \pi(a_t | s_t) \log \pi(a_t | s_t)$ is the entropy of policy π under state s_t .

Under the framework of maximum entropy reinforcement learning, the soft Q-function $Q^\pi(s_t, a_t)$ is defined as the expected sum of future discounted reward and entropies when the state s_t executes the action a_t and subsequently follows the policy π :

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \sum_{l=1} \gamma^l \mathbb{E}_{(s_{t+l}, a_{t+l}) \sim \rho_\pi} [r(s_{t+l}, a_{t+l}) + \mathcal{H}^\pi(\cdot | s_{t+l})] \quad (3)$$

Similar to standard reinforcement learning, the soft Bellman equation can be derived as:

$$Q^\pi(s_t, a_t) = R(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [\mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})] + \mathcal{H}^\pi(\cdot | s_{t+1})] \quad (4)$$

The previous works [22], [23] have made great progress in learning the maximum entropy policy by directly solving the optimal soft Q function. Next, we will extend the algorithm under the framework of maximum entropy reinforcement learning under MDP to handle partially observable spaces. Then the soft recurrent policy gradient and soft Bellman equation are combined to develop the soft recurrent policy gradient algorithm.

III. PROBLEM FORMULATION

In this section, we address the formulation of the navigation problem for UAVs in unknown complex environments. Firstly, we establish the dynamic model of the UAV, followed by a description of the objectives and metrics for UAV navigation.

A. UAV Model

The UAV in this paper is set as a quadrotor, and its model contains twelve states: the absolute position $(x, y$ and $z)$ in the world coordinate system, the Euler angle representing the UAV rotation $(\phi - \text{roll}, \theta - \text{pitch}, \psi - \text{yaw})$, velocities (v_x, v_y, v_z) and angular velocities (p, q, r) in the body coordinate system. The world coordinate system and the body coordinate system jointly represent the position, direction and movement of the UAV, which is called the internal state description of the UAV.

We ignore the momentum of the UAV during flight, assuming that the steering action takes effect immediately. The 2D modeling of the previous work [27] was extended to 3D in this work, so that the dynamics of the UAV was formulated as:

$$\begin{cases} \vartheta_{t+1} = \vartheta_t + \rho_t \\ \varphi_{t+1} = \varphi_t + \tau_t \\ x_{t+1} = x_t + v \times \cos(\vartheta_{t+1}) \cos(\varphi_{t+1}) \\ y_{t+1} = y_t + v \times \sin(\vartheta_{t+1}) \cos(\varphi_{t+1}) \\ z_{t+1} = z_t + v \times \sin(\varphi_{t+1}) \end{cases} \quad (5)$$

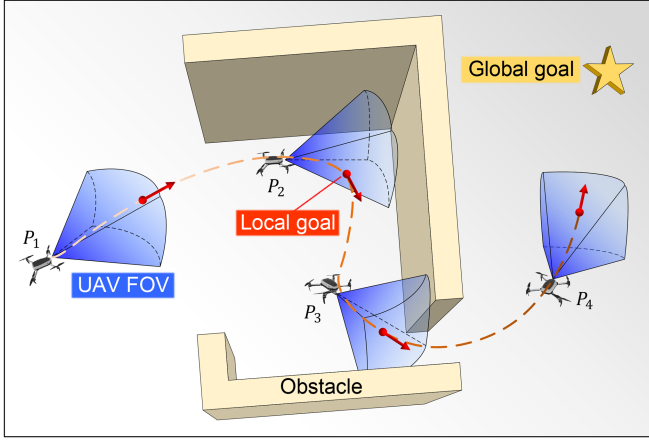


Fig. 1. Schematic illustration of a UAV with a limited field of view relying on local target guidance in an unknown environment.

where ϑ_t and φ_t are the direction angles of the UAV on the horizontal and vertical planes respectively, ρ_t and τ_t are the steering angles of the two angles respectively signal, v is UAV velocity.

B. Problem Description

This work aims to safely and efficiently navigate UAVs in unknown complex environments. Safety ensures collision-free navigation, while efficiency focuses on finding optimal paths respecting the UAV's dynamics. The primary challenge arises from dense obstacles and the absence of prior maps, prevalent in environments like forests or densely built urban areas. For example, in Fig. 1, the UAV is encouraged to move towards the target, but at the location of dense obstacles, denoted as P_2 , limited observability hinders finding a suitable direction, resulting in entrapment. The UAV's limited observability stems from cameras or range sensors with constrained field of view (FOV) in both horizontal and vertical directions, a realistic assumption. We assume accurate perception with an RGB-D camera or range sensor, and a static environment.

The objective of this paper is to enable a UAV to safely and smoothly navigate from an arbitrary starting point $p_0 = (x_0, y_0, z_0)$ to a target position $p_g = (x_g, y_g, z_g)$ in an unknown complex environment without collisions. At each time step t , the UAV receives environmental and pose information, collectively called observation o_t . The goal is to design a strategy that utilizes the UAV's observation o_t to generate local goals, which are used by the motion planner to generate dynamically feasible trajectories. This iterative process guides the UAV efficiently towards the target. As shown in Fig. 1, when the UAV perceives an obstacle ahead at location P_2 , the DRL-based policy generates a local goal $p_l = (x_l, y_l, z_l, \phi_l, \theta_l, \psi_l)$ (represented by a red dot for position and red arrow for direction) to navigate away from the predicament. The generation of local goals employs a mapless obstacle avoidance method, leveraging implicit environmental knowledge learned from historical trajectories using a recurrent neural network.

The navigation performance of the UAV is evaluated based on the success rate (reaching the target within a 0.5 m radius without collision), average time to reach the target (for successful cases), and average acceleration and jerk along the trajectory. All components of the proposed method should operate fully online and in real-time, ensuring their efficiency.

IV. HIERARCHICAL NAVIGATION FRAMEWORK

In this section, we first elaborate on the proposed RLoPlanner framework. Then the high-level recurrent soft policy gradient algorithm for generating local targets and the low-level motion planning algorithm for generating trajectories are introduced respectively.

A. RLoPlanner: RL With Local Planner

We model the UAV navigation in an unknown environment as a partially observable Markovian decision process. In the action space setting, this paper does not set the UAV's action set as the control signals of a continuously controlled UAV, such as velocity and angular velocity, as in previous work. This is because direct training of the UAV control signal makes the generated trajectories less smooth and slower to converge. Therefore, the consideration in RLoPlanner is to assume that the successful strategy can be described as a sequence of local targets: $\tau_{sus} = \{a_0, \dots, a_{\tau-1}\}$, each local target corresponding to the configuration space of the motion planner, i.e., representing the positional information in the 3D environment. The navigation algorithm should query the motion planner at each policy step to generate smooth and collision-free trajectories. The motion planner takes as input the local targets generated by the maximum entropy policy and outputs an action sequence that satisfies the dynamical feasibility. The RLoPlanner framework consists of two components, the local target generation policy and the local planner.

The local target generation policy accepts the raw sensor observation information o_t and outputs an action a_t , which is the local target of the motion planner. The combination of the trained local target generator policy and MP is a policy to solve the original POMDP. In summary, RLoPlanner lifts the original POMDP problem into a new formulation that can be more easily solved with reinforcement learning.

1) *Observation Space*: The local target generation strategy receives the raw sensor information and outputs the targets for the motion planner (as shown in Fig. 2). In order to successfully navigate to the target, the UAV needs to receive three aspects of observation information, that is, the UAV's own pose information, the relative relationship with the target, and the relationship with the environment. First, since the environment is unknown, the UAV cannot obtain its absolute position information. Therefore, we use the angle $[\vartheta]$ between the UAV's first viewing angle and the x-axis, and the angle $[\varphi]$ between the first viewing angle and the horizontal plane to describe its internal state, as shown in Fig. 3. In the real world, these two angles can be measured using a gyroscope. Secondly, the relationship between the UAV and the surrounding obstacles can be perceived using a variety of sensors. In order to narrow the gap from simulation

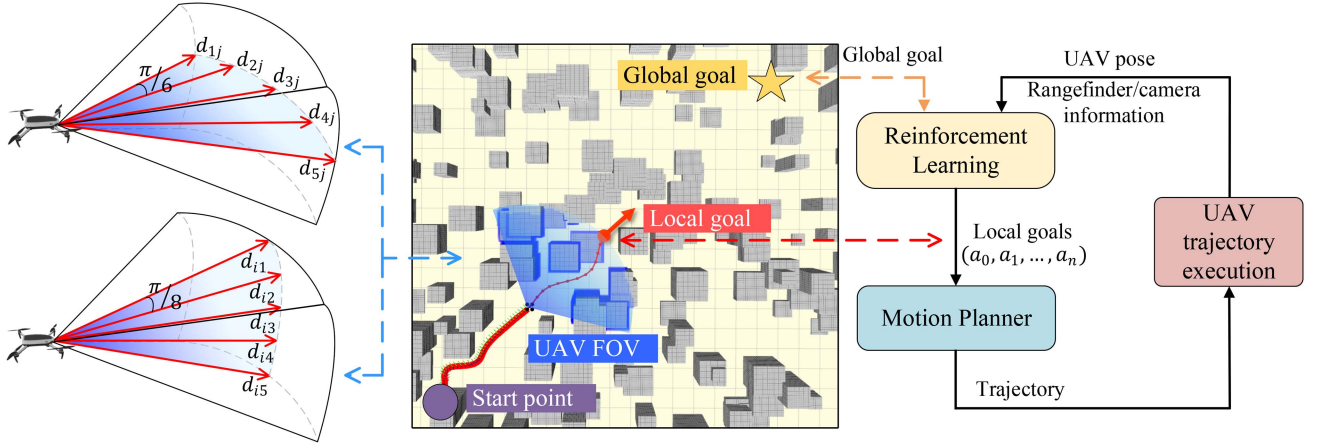


Fig. 2. Proposed method (RLoPlanner) consists of an upper DRL-based decision module and a motion planner-based planning module. The decision module predicts a local target based on current sensor information. This local goal is received by the motion planner-based planning module and generates a smooth, dynamically feasible trajectory to the local goal. This process repeats until the UAV reaches the final target.

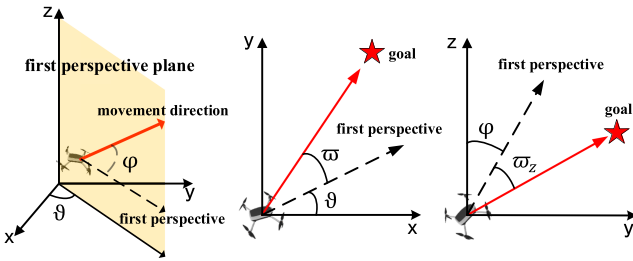


Fig. 3. Left figure in turn shows the perception of the rangefinder carried by the UAV on the horizontal plane. The middle and right figures show the schematic diagram of the rangefinder perception carried by the UAV on the vertical plane. The horizontal plane and the vertical plane contain a total of 25 sensing data.

to the real world, 25 ultrasonic rangefinders are used to perceive obstacles, expressed as $\mathcal{D} = \{d_{i0}, d_{i1}, d_{i2}, d_{i3}, d_{i4}\}_{i=1}^5$. Finally, the UAV needs to obtain the relative position and angle relationship with the target point. This information can be collected by a device similar to GPS, expressed as $\xi = [d_5, \varpi, \varpi_z]$. The above three kinds of observation information together constitute the observation space of UAV, namely $o = [\vartheta, \varphi, \mathcal{D}, \xi]$, where $\vartheta, \varphi, \varpi, \varpi_z \in [-\pi, \pi]$, $\{d_{i0}, d_{i1}, d_{i2}, d_{i3}, d_{i4}\}_{i=1}^5 \in [0, 10]$, and $d_5 \in [0, \infty]$. The output is the desired next local target.

In addition to the distance sensors for perceiving obstacle information, we also investigated a vision-based DRL navigation approach in order to make a reasonable comparison with traditional localization and planning methods. The training of vision-based DRL navigation strategies is computationally expensive due to the multiple matrix operations required by the convolutional neural network, and the convergence of the strategy becomes challenging due to the large number of images required [28]. Therefore, in the field of drone navigation, some studies tend to simplify images to achieve compressed information while preserving features [29]. Since models based on Variational Autoencoders (VAEs) [30] can typically extract rich and compact visual perception information from images, we attempted to combine VAE as an auxiliary role in unsupervised learning with policy-based DRL algorithms in this

study. We used VAE to compress image information and utilized the compressed image information as a state input in the training of the RSPG algorithm. This approach makes the network structure in the DRL algorithm more lightweight, enabling faster convergence of the drone's vision-based obstacle avoidance algorithm while utilizing fewer hardware resources. The input depth map is an 84×84 grayscale image with channel index 1, represented as $\mathcal{I}$. When the VAE converges, the latent vector generated in the VAE encoding part is a 50-dimensional code and retains the original depth map features. Using such latent vectors as states to participate in DRL training ensures that the states not only contain effective information from the original image but are also lightweight enough to enable fast convergence using simple fully connected layers. The observation space of the UAV constructed using this method is represented as $o = [\vartheta, \varphi, \mathcal{I}, \xi]$.

2) *Action Space*: The high-level observation regards the space of the local target as a continuous action space, and the policy network predicts a vector $a = [x, y, z, \phi, \theta, \psi]$, that is, a position information and direction information in the body coordinate system, which is used to represent the final end-effector pose after interaction. The variables ϕ, θ and ψ correspond to the direction information of the predicted local target point. This direction information is utilized to refine the motion trajectory. By considering this direction information, the lower-level motion planner can minimize unnecessary rotations to a maximum extent, resulting in more natural movements. The local target generation policy is trained by entropy-maximized policy gradients, and the resulting local targets are executed by the motion planner ego-planner introduced in the next section.

3) *Reward Design*: In order for the agent to gradually learn to make reasonable decisions about the observed information, it is necessary to choose an appropriate reward function as the feedback of the action. Design non-sparse rewards by combining prior knowledge in the navigation field, which includes distance rewards and obstacle penalties. The distance reward is expressed as:

$$r_{dist} = \omega_g (\|p^{t+1} - g\| - \|p^t - g\|) \quad (6)$$

where p^t represents the three-dimensional space coordinates of the UAV at t time, and g represents the three-dimensional space coordinates of the target point. This reward represents the shortened distance to the final goal after the UAV executes the trajectory to the local goal through the motion planner. With distance rewards, RL can be encouraged to choose local goals that are close to the end point. Besides, the selected local target should be in free space to keep the UAV safe, so an obstacle penalty is imposed to avoid collision, which is designed as:

$$r_{obs} = -e^{-\sigma d_{min}} \quad (7)$$

where d_{min} represents the Euclidean distance between the decision-making local target and the nearest obstacle.

The total reward is expressed as:

$$r = r_{dist} + r_{obs} \quad (8)$$

B. Recurrent Soft Policy Gradient

In order to enable the UAV to have a stronger exploration capability in an unknown environment to enable a higher success rate of reaching the final goal, the local goal generation policy should have randomness [31]. Therefore, this paper resorts to the maximum entropy policy gradient algorithm, where the addition of maximum entropy can keep the policy as random as possible while successfully completing the task. In this section we extend deep soft policy gradient (DSPG) algorithm [24] to handle partially observable spaces, and finally derive the recurrent soft policy gradient (RSPG) algorithm that includes memorability. The core idea of the DSPG algorithm for fully observable situations is that $\pi(a | s) = \mathbb{P}[a | s; \theta]$ represents a policy, this distribution randomly chooses an action a in state s according to the parameter vector. Policy updates follow:

$$\begin{aligned} \nabla_{\theta} J(\pi_{\theta}) &= \mathbb{E}_{(s,a) \sim \rho_{\pi_{\theta}}} [(Q^{\pi}(s,a) - \log \pi(a | s) \\ &\quad - 1) \nabla_{\theta} \log \pi(a | s)] \end{aligned} \quad (9)$$

where the expected value corresponds to the amount of state visits. In practice, consider that $Q^{\pi}(s,a)$ is considered to be approximated by a learned function approximator.

In partially observable cases, the optimal policy and associated action-value functions are functions based on the entire previous history h_t . The main change we introduce is the use of recurrent neural networks to allow the network to learn to preserve relevant past information, which is necessary to solve POMDP. Recurrent neural networks leverage internal memory to handle sequential data and retain information over multiple time steps. This is achieved by propagating the hidden state to the subsequent time step. However, training RNNs can be challenging due to the issue of gradient explosion or vanishing gradient, where the gradient either grows exponentially or diminishes rapidly at each time step. To address this challenge, we employ an LSTM network [32], which incorporates gate mechanisms to regulate the flow and retention of information. Unlike a traditional fully connected layer, LSTM networks excel at learning long-term dependencies, making them suitable for mitigating the gradient-related problems in training recurrent neural networks.

Therefore, the relationship between the observed historical trajectory h_t and the potential real state s_t is expressed as $s_t \sim p(s_t | h_t)$. Taking the expectation about the state distribution $s_t \sim p(s_t | h_t)$, the soft Q function under the maximum entropy reinforcement learning framework can be defined as:

$$Q^{\pi}(h_t, a_t) = \mathbb{E}_{s_t | h_t} [r(s_t, a_t) + \sum_{l=1}^{\infty} \gamma^l \mathbb{E}_{(s_{t+l}, a_{t+l}) \sim \rho_{\pi}} [r(s_{t+l}, a_{t+l}) + \mathcal{H}^{\pi}(\cdot | s_{t+l})]] \quad (10)$$

Soft policy gradient adjusts the policy parameters towards a greater entropy regularized expected return objective by sampling a random policy.

Theorem 1: (Recurrent Soft Policy Gradient) Considering learning a maximum entropy policy for a POMDP with a continuous space, the policy update follows:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau} [(Q^{\pi}(h, a) - \log \pi(a | h) - 1) \nabla_{\theta} \log \pi(a | h)] \quad (11)$$

where the expectation of the entire trajectory $\tau = (s_1, o_1, a_1, s_2, o_2, a_2, \dots)$ is drawn from the trajectory distribution of the current policy $\pi : p(s_1)p(o_1 | s_1)\pi(a_1 | h_1)p(s_2 | s_1, a_1)p(o_2 | s_2)\pi(a_2 | h_2) \dots$, and the historical trajectory is $h_t = (o_1, a_1, o_2, a_2, \dots, a_{t-1}, o_t)$.

Proof: According to the definition of soft Q function under POMDP, the entropy regularized expected reward target can be written as:

$$J(\pi_{\theta}) = \mathbb{E}_{(h,a) \sim \rho_{\pi_{\theta}}} [Q^{\pi}(h, a) + \mathcal{H}^{\pi}(\cdot | h)] \quad (12)$$

In order to calculate the gradient of the above formula, first derive the entropy regularization expected reward target $J(\pi_{\theta} | h_0)$ from the initial state s_0 gradient.

Then we have

$$\begin{aligned} \frac{\partial J(\pi_{\theta} | h)}{\partial \theta} &= \frac{\partial}{\partial \theta} \mathbb{E}_{a \sim \pi_{\theta}} [Q^{\pi}(h, a) + \mathcal{H}^{\pi}(\cdot | h)] \\ &= \frac{\partial}{\partial \theta} \left[\sum_a \pi(a | h) Q^{\pi}(h, a) + \mathcal{H}^{\pi}(\cdot | h) \right] \\ &= \mathbb{E}_{(h_t, a_t) \sim \rho_{\pi}} [\nabla_{\theta} \log \pi(a_t | h_t) Q^{\pi}(h_t, a_t)] \\ &\quad + \nabla_{\theta} \mathbb{E}_{h_t \sim \rho_{\pi}} [\mathcal{H}^{\pi}(\cdot | h)] \end{aligned} \quad (13)$$

where $(h_t, a_t) \sim \rho_{\pi}$ represents the observed dynamics of the historical trajectory.

The gradient of entropy is:

$$\begin{aligned} \nabla_{\theta} \mathcal{H}(\pi) &= -\nabla_{\theta} \mathbb{E}_{h_t \sim \rho_{\pi}} [\mathbb{E}_{a_t \sim \pi(a_t | h_t)} [\log \pi(a_t | h_t)]] \\ &= -\mathbb{E}_{(h_t, a_t) \sim \rho_{\pi}} [\nabla_{\theta} \log \pi(a_t | h_t) \log \pi(a_t | h_t) \\ &\quad + \nabla_{\theta} \log \pi(a_t | h_t)] \\ &= -\mathbb{E}_{(h_t, a_t) \sim \rho_{\pi}} [\nabla_{\theta} \log \pi(a_t | h_t) \\ &\quad \times (1 + \log \pi(a_t | h_t))] \end{aligned} \quad (14)$$

The gradient can be obtained as:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau} [(Q^{\pi}(h, a) - \log \pi(a | h) - 1) \nabla_{\theta} \log \pi(a | h)] \quad (15)$$

■

In practice, we replace the true value Q^π with the learned approximation Q^ω . Therefore, instead of directly conditioning on the entire history of observations, we effectively train a recurrent neural network to summarize historical information using backpropagation over time. Therefore, we can obtain the soft policy gradient approximated by the following function:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_\tau [(Q^\omega(h, a) - \log \pi(a | h) - 1) \nabla_\theta \log \pi(a | h)] \quad (16)$$

The proposed algorithm is shown in Algorithm 1, where two target actors and a critic network are used to stabilize the learning process. When applying the recurrent soft policy gradient, we use the gradient clipping method to solve the problem that the soft Q function will affect the gradient scale. Specifically, we use a global specification $\mathcal{N}$ to constrain the soft policy gradient in an appropriate range. In addition, the recurrent soft policy gradient depends on the state distribution and the current policy, so we use a double sampling method to estimate the expected value of the state and action separately.

The RSPG algorithm framework includes an actor network with LSTM, a critic network with LSTM, and an experience replay mechanism. It takes initial state observation h_0 as input and generates a stochastic action policy $\pi^\theta(h_t)$ during action selection. The agent produces a local goal action a_t based on current state observation o_t and evaluates it with reward r_t . Historical and current information $\{h_{t-1}, a_t, o_t, r_t\}$ is stored in the experience pool. Random sampling of historical trajectories from the pool replaces the expectation in (11), facilitating iterative updates of network parameters through stochastic gradient descent. This leverages the efficiency improvement of experience replay, enabling an action policy maximizing replay and reaching the target. The RSPG algorithm includes an online network and a target network. The online network's actor network outputs a random action policy π with parameters ω , while the critic network calculates Q-value evaluation parameters as θ . The target network shares the same structure and uses soft update with parameters θ' and ω' , delaying updates to θ and ω . The inclusion of a target network enhances algorithm stability.

C. Ego-Planner With Local Target Cost

After the RL-based upper-level algorithm selects the local target point, the lower-level motion planning algorithm will calculate a feasible trajectory based on the relative pose relationship between the UAV and the local target point at the current moment. The trajectory needs to satisfy the constraints of system dynamics and smooth control, so that the algorithm can be easily deployed to the real world. The trajectory should satisfy safe obstacle avoidance, so obstacles within the range of the sensor should be taken into consideration. To ensure a collision-free smooth path, the ESDF-free gradient-based planning framework proposed by Zhou et al. [33] is used in this work to optimize the spline trajectory. This method represents the trajectory generation as a nonlinear optimization problem, does not pre-establish the Euclidean Signed Distance Field (ESDF), and only calculates the gradient when the trajectory encounters a new obstacle. If the generated trajectory violates the dynamic constraints, the trajectory time is reallocated during optimization

Algorithm 1: RSPG.

- 1: Initialize critic network $Q^\omega(a_t, h_t)$ and actor $\pi^\theta(h_t)$ with parameters ω and θ .
 - 2: Initialize target critic networks $Q^{\omega'}(a_t, h_t)$ and target actor network $\pi^{\theta'}(h_t)$ with weights $\omega' \leftarrow \omega, \theta' \leftarrow \theta$.
 - 3: Initialize replay buffer R .
 - 4: **for** episodes = 1, M **do**
 - 5: Initialize a stochastic process N for action exploration
Receive an initial observation o_0 (or history trajectory h_0)
 - 6: **for** t = 1, T **do**
 - 7: Sample an action $a_t = \pi^\theta(\cdot | h_t)$
Execute action a_t , obtain reward r_t and observation o_t
Store transition (h_{t-1}, a_t, o_t, r_t) into R
Update history trajectory $h_t = [h_{t-1}, o_t, a_t]$
Sample a minibatch of N transition $\{(h_t^i, o_t^i, a_t^i, r_t^i)\}_{i=1}^L$
Sample M actions a_t^i for each history trajectory h_t^i using $\pi^\theta(\cdot | h_{t+1})$
Set the sampled soft Bellman backup operator:
$$\tilde{T}Q^\omega = r_t + \frac{\gamma}{M} \sum_t [Q^\omega(h_{t+1}, a_{t+1}) - \log \pi_{\theta'}(a_{t+1} | h_{t+1})]$$

Update the critic by minimizing the soft loss:
$$\mathcal{L}(\omega) = \frac{1}{N} \sum_i [(Q^\omega(h_i, a_i) - \tilde{T}Q^\omega)^2]$$

Sample M actions a_t^i for each history trajectory h_i using $\pi^\theta(\cdot | h_t)$
Update the actor policy using the clipped soft policy gradient:
$$\Delta\theta \approx \frac{1}{NT} \sum_i \sum_t [(Q^\omega(h_t^i, a_t^i) - \log \pi(a_t^i | h_t^i) - 1) \nabla_\theta \log \pi(a_t^i | h_t^i)]$$

Update the target networks:
$$\omega' \leftarrow \tau\omega + (1 - \tau)\omega'$$

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$$
 - 8: **end for**
 - 9: **end for**
-

to generate new B-splines. By omitting the maintenance of ESDF, the whole method shortens the trajectory calculation time by an order of magnitude compared with the traditional pre-build ESDF methods, and at the same time satisfies the smoothness and security of the trajectory.

A Uniform B-splines with N_c control points $\{\mathbf{Q}_0, \mathbf{Q}_1, \dots, \mathbf{Q}_N\}$ be used to compare The trajectory of the man-machine is parameterized, and the cost function of the trajectory optimization problem is as follows:

$$\min_Q J = \lambda_1 J_s + \lambda_2 J_c + \lambda_3 J_d + \lambda_4 J_{lp} \quad (17)$$

where J_s is a cost function that guarantees a smooth trajectory by penalizing the square of the trajectory, J_c is a collision function that punishes the minimum distance between the UAV and obstacles, J_d is a cost function that guarantees dynamical feasibility by limiting the higher-order derivatives of the trajectory in each dimension. For specific parameter settings and function properties, please refer to [33]. $\lambda_{1,2,3,4}$ is the weight parameter of each penalty term. In addition, this paper also adds a local target cost function J_{lp} , whose purpose is to make the trajectory close to the local target generated by the upper reinforcement learning algorithm. This can be achieved by penalizing deviations in position and velocity at the end of the trajectory from the expected value of the local target. Like the collision cost function J_c and the dynamic feasibility function J_d , the cost function is expressed in the form of soft constraints, and the function is defined as follows:

$$J_{lp} = \lambda_p (p(t_{lp}) - p_{lp})^2 + \lambda_v (p'(t_{lp}) - v_{lp})^2 \quad (18)$$

where t_{lp} is the end time of the local trajectory, $p(t)$ is the optimized trajectory, p_{lp} and v_{lp} are the position and velocity of the desired local target, λ_p and λ_v are weight parameters.

The addition of this cost function improves the feasibility of the optimized trajectory, and the impact of this item on the navigation success rate will be analyzed in the experiment.

It should be emphasized that this paper uses EGO-planner for computational efficiency, but the hierarchical modular design can easily replace other gradient-based trajectory planning algorithms. The control command generated by the trajectory planner is executed on the system within T seconds, and then the airborne sensor gets a new image (or distance information), and then generates a new local target and trajectory planning. The whole process is repeated until the UAV reaches the final target.

V. SIMULATION EXPERIMENTS

In this section, we first introduce the simulated experimental environment, then evaluate the proposed recurrent soft policy gradient algorithm, and finally compare the RLoPlanner navigation framework with end-to-end reinforcement learning navigation algorithms and traditional mapping and planning methods.

A. Simulation Setup

The simulated environment, as shown in Fig. 4, is generated based on the mockmap in the ROS environment. It encompasses four distinct environments, namely randomly generated square pillars, columns, circles, and irregularly shaped foggy 3D obstacles, effectively simulating urban buildings, forest trees, and other relevant obstacles. Each map exhibits dimensions of $40 \times 40 \times 12$ meters, with the obstacle area confined within a $35 \times 35 \times 12$ meters region, ensuring the safety and unobstructed traversal of the UAV from its designated start to endpoint.

To quantitatively assess obstacle density, the metric employed is the percentage of occupied volume within the space. As the volume of obstacles increases, it poses an escalating challenge to the UAV's navigation and exploration capabilities. This is

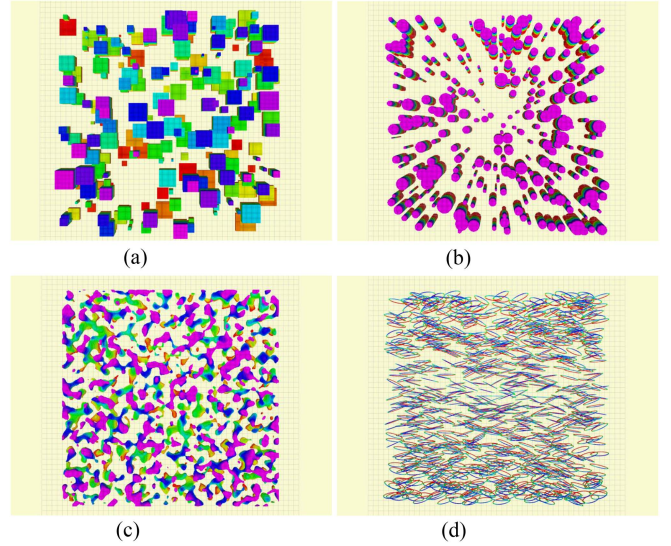


Fig. 4. Navigation algorithm will be tested in the following four environments, and trained by randomly selecting scenes and starting point pairs. The environment includes square column obstacles, cylindrical obstacles, simulated forest obstacles, and circular obstacles. (a) Scene-I. (b) Scene-II. (c) Scene-III. (d) Scene-IV.

due to the interconnectivity observed among adjacent obstacles, leading to the formation of numerous localized traps, vividly illustrated in Fig. 1. A comprehensive evaluation of algorithm performance is conducted across various obstacle density scenarios, with specific emphasis on higher obstacle densities that better align with real-world environments. This focused analysis allows for a more accurate assessment of algorithmic effectiveness and adaptability in challenging conditions. All simulations were executed on a computing platform operating Ubuntu 16.04, equipped with 32 GB of RAM, an AMD R5-5600 G CPU, and a GeForce GTX 1080 GPU. The UAV underwent training for a total of 10,000 time steps, approximately equivalent to a 12-hour execution duration.

The hyperparameters in the model are tuned through a series of repeated experiments, and we deploy the model with the best hyperparameters. The maximum speed limit of UAV is 2.1 m/s , and the flight altitude limit in the environment is $[0.0 \text{ m}, 10.0 \text{ m}]$. The parameters of the reward are set as: $\sigma = 2.0$, $\alpha = 8.0$, $\beta = 25.0$. The longest history estimate length is limited to 100. The experiment assumes that the learning rate of Adam Optimizer is 0.01, and the update rate of the target network is $\tau = 0.01$. The replay buffer has a size of 10^5 and is sampled every 100 time steps. The discount factor is $\gamma = 0.99$. The training network structure is shown in Fig. 5.

B. Comparisons Baselines and Metrics

We conduct a comprehensive comparison between the proposed approach and three major classes of approaches: the end-to-end RL approach, the combination of DRL with motion planner approach, and the traditional planner approach.

The first class, the end-to-end RL approach, involves the direct mapping of raw perception to control signals for navigation.

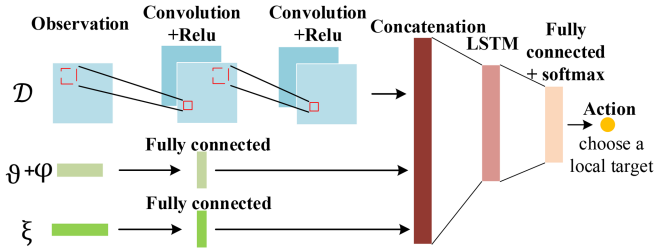


Fig. 5. Architecture of actor networks to select actions.

We compare our approach with four methods in this category: FRDPG, RSVG(0), DSPG, and RSPG. FRDPG [16] is a deterministic policy gradient algorithm that accelerates convergence by updating parameters based on historical trajectories rather than the entire trajectory. It has demonstrated successful UAV navigation in large-scale complex environments. The RSVG(0) algorithm is a recursive version of the stochastic value-gradient algorithm that was used in [27] for point-to-point exploration navigation tasks by UAVs in unknown complex environments. The observation information for these four end-to-end algorithms follows the design presented in the paper. It utilizes a rangefinder, configured similarly to RLoPlanner(rangefinder), to sense obstacle information in the environment, and an IMU to detect positional changes of the UAV. The target position is not dependent on GPS but is inputted into the UAV's airframe coordinate system at the same initial moment as RLoPlanner. The action space for the end-to-end RL method is derived from control signals, utilizing a vector $a = [\rho, \varphi]$ where ρ represents the throttle signal and φ denotes the steering signal within the range of $-\pi$ to π . These two signals constitute the UAV's action space. In addition to the reward settings similar to RLoPlanner(rangefinder), the reward signal in this method also incorporates control over the throttle change amount $\Delta\rho$ to avoid abrupt power signal changes.

In addition to the four end-to-end RL methods mentioned above, we also compare a framework structure similar to the RLoPlanner algorithm. This framework employs a DRL-based approach to generate a generation policy for a local target and employs a local planner to reach that local target continuously until the global target is reached. However, in our comparison, the upper layer algorithm in RLoPlanner(rangefinder), RSPG, is replaced with two algorithms: RDPG and RSVG(0). The lower layer utilizes ego-planner for local planning. The observation space, action space, and reward signal for these three algorithms are configured similarly to RLoPlanner(rangefinder). By comparing these four RL methods that incorporate the same local planning approach, we can highlight the effectiveness of different DRL algorithms in formulating local goal policies.

Furthermore, we compare our approach with mapping and planning approaches. To emphasize the distinction between our proposed algorithm and mapping-planning algorithms for local target decision-making, we adopt an open-source vision-based planning and flying scheme for UAVs [33]. This scheme encompasses global trajectory generation and safe local planning. We utilize its lightweight ESDF map for map information maintenance, and employ the A* algorithm for global planning

and ego-planner for local planning. In our paper, we set the input observation information for this method as RGB-D with a limited field of view, making it suitable for comparison with the RLoPlanner(image) method, which also utilizes depth map information within the same observation range. Additionally, this algorithm estimates the airframe position by fusing visual odometry information with IMU data. The overall path planning algorithm is divided into global planning and local planning. The A* algorithm used for global planning adopts an optimistic approach, treating unknown environments as unoccupied. This is implemented by setting a large radius for unknown space and considering the unknown environment within it as free space. A segment of the path planned by the A* algorithm is generated from the starting point, and the local trajectory is continuously replanned using the ego-planner as the UAV progresses. If the local planner becomes trapped, a new path is generated using the globally planned A* algorithm in an attempt to escape the dilemma.

The navigation methods are compared with the following metrics:

- Average Reward:* We use the learned policy on 200 navigation tasks in four environments, using the average reward over all time and agents as the reward. Smaller rewards mean poorer performance.
- Success rate:* The number of UAVs reaching the target point without collision as a percentage of the total.
- Trapped rate:* The number of UAV agents that did not reach the target point within the time limit as a percentage of the total.
- Collision rate:* The number of UAV collisions with obstacles resulting in mission failure as a percentage of the total number of agents.
- Path efficiency:* Path efficiency is determined by calculating the ratio of the sum of straight-line distances between the UAV's starting point and target point to the sum of the actual trajectory length of the UAV, considering only the cases where navigation was successful.
- Average acceleration and average jerk:* These metrics assess the performance in terms of smoothness and energy consumption during the navigation task. Average acceleration measures the average rate of change of velocity over time, while average jerk quantifies the average rate of change of acceleration over time.

C. Performance of Maximum Entropy Policy Gradient Algorithm

The average training rewards for the FRDPG, RSVG(0), DSPG, and RSPG algorithms are depicted in Fig. 6. To obtain these results, we computed the cumulative rewards accumulated by the agent over 250 training episodes and then averaged them. In the following sections, we provide a detailed analysis of the convergence curves for our proposed RSPG algorithm, comparing its performance to the other three algorithms individually. By examining these convergence curves, we gain valuable insights into the convergence behavior and effectiveness of each algorithm.

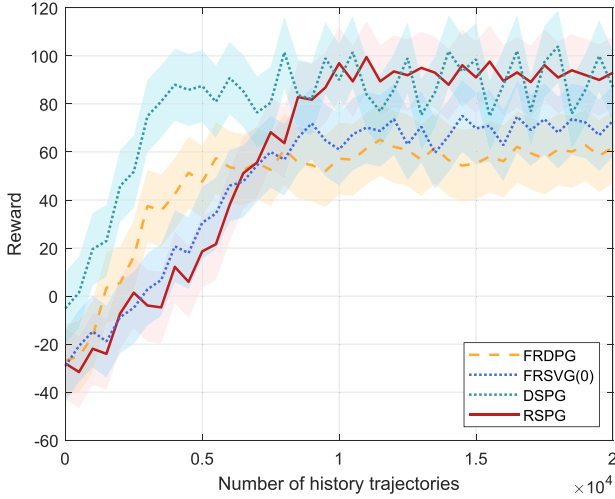


Fig. 6. Reward during all the training episodes in Scene-I.

a) *Comparison with DSVG*: As can be seen from Fig. 5, compared with three algorithms, FRDPG, FRSVG(0) and RSPG, which include recurrent networks, the DSPG algorithm reaches convergence around 5000 episodes, which is the fastest one among the four algorithms. And the DSPG algorithm has a higher average total return after convergence than the FRDPG algorithm and the FRSVG(0) algorithm without the addition of the maximum entropy policy, thanks to the addition of the maximum entropy policy, which allows the intelligence to increase its ability to explore so as not to make the intelligence commit to a policy prematurely and converge to suboptimal behavior. Our proposed RSVG algorithm reaches convergence slower than the DSPG algorithm, reaching convergence in about 9000 episodes, but outperforms the DSPG algorithm in terms of stability of convergence. The recurrent network structure ensures the convergence stability of the FRDPG algorithm, the RSVG(0) algorithm, and our proposed RSPG algorithm.

b) *Comparison with FRDPG and FRSVG(0) algorithms*: The FRDPG algorithm reaches convergence about 3500 episodes earlier than the RSPG algorithm due to its fast policy update approach. However, its average total return after convergence is much lower than that of the RSPG algorithm, this due to its deterministic policy gradient algorithm makes the intelligence will quickly converge to a single action when exploring the task and cannot acquire more valuable behaviors. Both the RSPG and FRSVG(0) algorithms exhibit similar convergence rates initially, taking approximately 9000 rounds to reach convergence. However, once converged, the average payoff of the FRSVG(0) algorithm is lower than that of the RSPG algorithm. While both algorithms aim to enhance exploration ability, there is a distinction in their overall training objectives. The FRSVG(0) algorithm focuses solely on rewards, while the RSPG algorithm incorporates a maximum entropy policy directly into the training objective. This inclusion of the maximum entropy policy results in a more random policy and ultimately leads to a higher average reward.

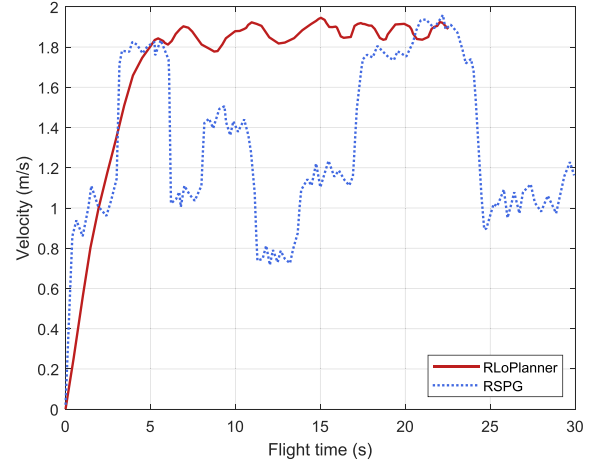


Fig. 7. Schematic diagram of the navigation trajectory of the RLoPlanner agent trained in the simulation environment. The blue circles and red pentagrams represent the starting point and target point of an arbitrary choice, respectively. The curve connecting it represents the trajectory of the UAV.

D. RLoPlanner Compared With End-to-End RL Methods

a) *Compared with FRDPG and FRSVG(0)*: We begin by comparing the navigation performance of the proposed navigation framework, RLoPlanner(rangefinder), with two end-to-end algorithms: FRDPG and FRSVG(0). These algorithms differ in that FRDPG adopts a deterministic policy, while the FRSVG(0) algorithm utilizes a stochastic policy. Both FRDPG and FRSVG(0) algorithms have been validated and have shown good performance in the field of UAV navigation, with a navigation success rate exceeding 96.75%. These results were obtained from an average of 250 navigation tests conducted in four simulated environments.

Both end-to-end algorithms achieve similar collision and entrapment rates as the proposed RLoPlanner(rangefinder) method, but they perform worse in terms of trajectory generation for navigation. Qualitatively, it can be observed from the navigation example plots that the navigation trajectories generated by the FRDPG and FRSVG(0) algorithms contain more inflection points and discontinuities, posing significant challenges for controlling actual UAVs. Table I shows that the flight time of the UAV running the FRDPG algorithm is approximately 31.84 seconds, while the flight time of the UAV running the FRSVG(0) algorithm is even longer, at 34.12 seconds. This is nearly 50% longer than the flight time of the UAV using the proposed RLoPlanner(rangefinder) algorithm. Additionally, the average acceleration and jerk of the FRDPG and FRSVG(0) algorithms are two to three times higher than those of the RLoPlanner(rangefinder) algorithm. This implies that both end-to-end algorithms experience frequent sharp braking and rapid throttle changes during flight, which is unfavorable for energy efficiency and flight performance of the UAV. This observation is supported by Fig. 7, which displays the speed control curve. Despite incorporating a penalty term to limit throttle variation, the speed control curve of the E2E method remains discontinuous and abrupt during testing.

TABLE I

DATA TABLES OF SUCCESS RATE, COLLISION RATE, TRAPPING RATE, FLIGHT TIME, ACCELERATION AND JERK OBTAINED AFTER 250 EXPERIMENTS AVERAGED BY VARIOUS NAVIGATION ALGORITHMS IN A RANDOM ENVIRONMENT. BOLD FONT INDICATES THE BEST PERFORMANCE

Category	Metric	Input	Success rate(%)	Collision rate(%)	Trapped rate(%)	Flight time (s)	Acceleration (m/s^2)	Jerk (m/s^3)
E2E	FRDPG	Rangefinder+IMU	96.75	1.75	1.50	31.84±5.06	0.49±0.03	8.77±0.57
	FRSVG(0)	Rangefinder+IMU	97.25	2.00	0.75	34.12±4.56	0.55±0.02	7.43±0.49
	DSPG	Rangefinder+IMU	74.75	9.50	15.75	37.71±5.83	0.53±0.05	9.84±0.61
	RSPG	Rangefinder+IMU	95.75	2.50	1.75	29.96±4.39	0.46±0.04	7.18±0.52
RL with planner	FRDPG with ego-planner	Rangefinder+IMU	96.5	2.00	1.50	21.65±3.48	0.15±0.01	3.52±0.15
	FRSVG(0) with ego-planner	Rangefinder+IMU	97.50	1.75	0.75	20.38±3.57	0.14±0.01	3.72±0.14
	DSPG with ego-planner	Rangefinder+IMU	77.25	8.50	14.25	22.14±3.96	0.13±0.02	3.64±0.16
	RLoPlanner(rangefinder)	Rangefinder+IMU	97.50	1.50	1.00	19.9±3.72	0.14±0.01	3.65±0.13
	RLoPlanner(image)	RGB-D+IMU	96.75	1.50	1.75	23.36±3.51	0.16±0.02	3.89±0.17
Classic planner	A* with ego-planner	RGB-D+IMU	78.5	6.25	15.25	20.83±2.97	0.13±0.01	3.43±0.14

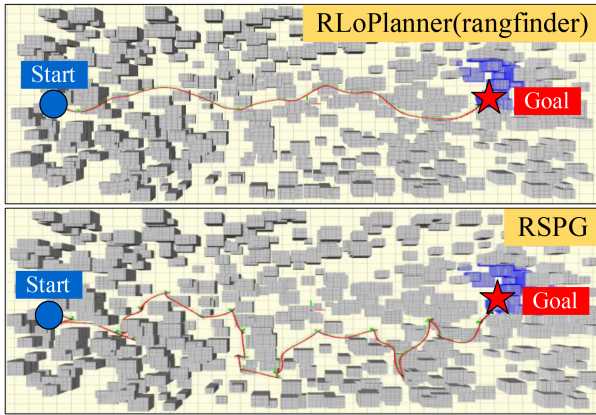


Fig. 8. Schematic diagram of the navigation trajectory of the RLoPlanner (rangfinder) and RSPG algorithms in the environment SCENE-I. The blue circle and the red pentagram represent the starting point and the target point, respectively.

b) Compared with DSPG and RSPG: Based on Table I, it can be observed that the end-to-end DSPG algorithm has an average navigation success rate of only 74.75%, which is the lowest among the four end-to-end DRL algorithms. The main reason for navigation failures is that in many episodes, the UAV executing the DSPG strategy experienced collisions and entrapment, with entrapment occurring in the majority of cases. Upon analysis, it is found that the DSPG method is more suitable for fully observable environments, and in partially observable domains, it cannot gather information from historical trajectories, resulting in a high number of instances where it becomes trapped in local difficulties. Additionally, its flight performance is inferior to the proposed RLoPlanner(rangefinder) algorithm. In successful navigation episodes, the DSPG algorithm takes approximately 18 seconds longer in flight time compared to the RLoPlanner(rangefinder) algorithm. The occurrence of this result is attributed to the fact that end-to-end learning algorithms not only need to learn how to navigate the UAV to the target but also how to control the UAV's actions, which presents a challenge to the algorithm. Fig. 8 shows the navigation trajectories of RLoPlanner (rangfinder) and RSPG algorithms.

The navigation performance of the RSPG algorithm has shown significant improvement compared to the DSPG algorithm, with an increase of nearly 20% in navigation success rate. It also exhibits the shortest flight time among the four end-to-end DRL methods, and the average acceleration and jerk

are relatively low. This indicates that the recursive network of the RSPG algorithm enhances the agent's ability to summarize information, leading to improved performance in navigation tasks in complex and partially observable environments. The proposed RLoPlanner(rangefinder) algorithm combines a local planner with the RSPG algorithm, resulting in a significantly enhanced trajectory generation capability compared to the pure RSPG algorithm. Flight time is further reduced, and the trajectories are smoother and more energy-efficient.

E. Comparison With the DRL Combined Planner Methods

To more fairly compare the performance of different algorithms for local goal policies, in this section we compare RLoPlanner(rangefinder) with two other DRL combined with local planner approaches, which are FRDPG with ego-planner and FRSVG(0) with ego-planner, respectively. In addition to comparing the above UAV the success rate, collision rate and trapped rate of performing navigation tasks, we deliberately selected scenario 1, which presents the greatest challenge to the algorithm. This scenario is unique because when the density of obstacles increases, the cylindrical obstacles connect to form wall-like structures and other complex configurations. Such partially observable environments with limited visibility impose higher demands on the navigation capabilities of the UAV. This also helps distinguish the differences in local goal selection policies among different algorithms.

From Fig. 9, it can be observed that under low obstacle density conditions, both the FRDPG with ego-planner and FRSVG(0) with ego-planner algorithms perform well, showing improvements compared to the end-to-end FRDPG and FRSVG(0) approaches. Until the obstacle density reaches 50%, the success rate of both algorithms can be maintained at around 93%. However, as the obstacle density increases, the advantages of the proposed RLoPlanner(rangefinder) algorithm gradually become evident, with a higher success rate compared to the other two methods. When the obstacle density reaches 70%, its success rate can still maintain around 92% and the trapped rate can be lower than 4%.

In summary, our RSPG method with motion planner outperforms the other two algorithms. Firstly, it reduces the trapped rate during navigation compared to DSPG with motion planner, thanks to our recurrent network summarizing historical trajectory data and enhancing local goal generation. Secondly,

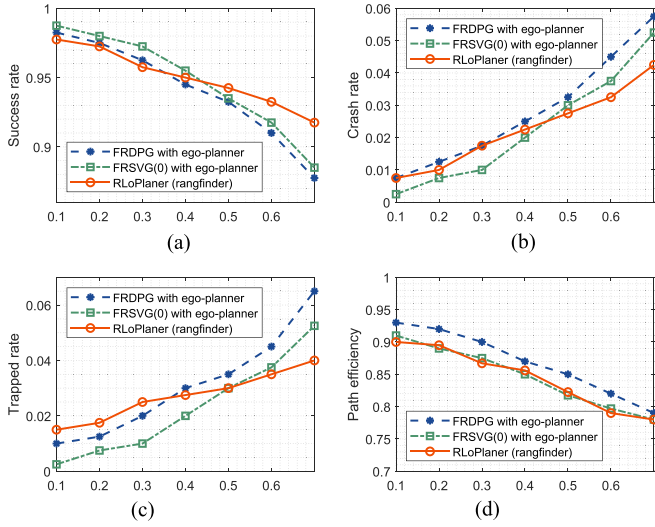


Fig. 9. Schematic diagram of the proposed RLoPlanner and similar DRL algorithms combined with motion planner structures for changes in success rate, collision rate, trapped rate, and path efficiency under different obstacle densities. (a) Obstacle density. (b) Obstacle density. (c) Obstacle density. (d) Obstacle density.

FRSVG(0) with motion planner closely resembles our RLoPlanner(rangefinder) algorithm, both using recurrent networks and stochastic policies to boost UAV exploration in unknown environments. However, FRSVG(0) lacks a separate value network, resulting in less stable training outcomes than ours. Lastly, our maximum entropy policy, compared to FRDPG with motion planner, encourages more exploratory UAV behavior during training, preventing premature convergence. Thus, RLoPlanner(rangefinder) achieves a higher success rate, particularly in dense obstacle environments.

F. Comparison With Mapping and Planning Methods

In this section, we compare RLoPlanner(image) with A* with ego-planner, both using an onboard camera for RGB-D input. We conducted benchmark tests in the same simulation scenario, and Table I summarizes the results. RLoPlanner(image) achieves a navigation success rate approximately 20% higher than the mapping-and-planning method, with only a one-ninth trapping rate. This demonstrates RLoPlanner(image)'s significant reduction in the likelihood of local traps. This improvement is attributed to the upper-layer recurrent network in the RSPG algorithm, summarizing historical information and aiding in avoiding choices leading to local dilemmas during goal selection. Both methods show similar performance in average acceleration, average jerk, and the overall navigation task completion time, approximately 20 seconds. This similarity arises from the use of the same local planner for trajectory generation and optimization, resulting in smoother and more energy-efficient velocity changes in the generated trajectories.

We also compared the navigation performance of RLoPlanner(image) and A* with ego-planner algorithms in unknown environments under different obstacle densities, and the results are shown in Fig. 10. It can be observed that as the obstacle

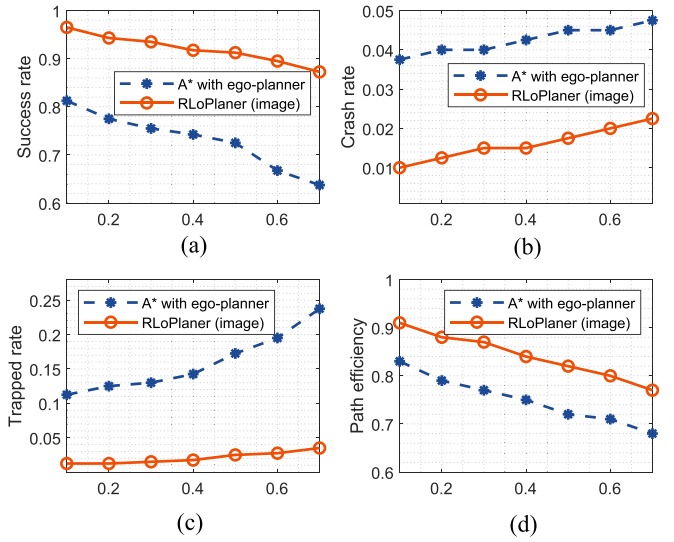


Fig. 10. Schematic diagram of the changes in the success rate, collision rate, trapped rate and path efficiency of the proposed RLoPlanner and the classic motion planning algorithm under different obstacle densities. (a) Obstacle density. (b) Obstacle density. (c) Obstacle density. (d) Obstacle density.

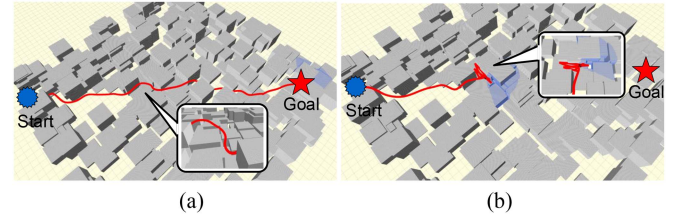


Fig. 11. Navigation diagram of RLoPlanner(image) and A* with ego-planner methods in Scene-1 with an obstacle density of 60%. (a) RLoPlanner(image). (b) A star with ego-planner.

density increases, the navigation success rate of the A* with ego-planner algorithm continuously decreases. Once the obstacle density exceeds 50%, the navigation success rate of the A* with ego-planner algorithm drops below 70%. Correspondingly, the trapped rate of the A* with ego-planner algorithm increases with higher obstacle densities, and the denser the obstacles, the faster the trapping rate increases. When the obstacle density reaches 70%, the trapped rate reaches nearly 25%. This is because the A* search algorithm is susceptible to local minima and short-sighted planning, leading to the UAV getting trapped easily. On the other hand, our proposed RLoPlanner(image) algorithm demonstrates good navigation performance even in dense obstacle environments.

Increasing obstacle density in the experimental environment creates dead-end-like challenges, particularly testing the navigation and active escape abilities of UAVs with limited perception. The key to escaping these dead ends is selecting local target points. Effective local target point generation guides UAVs toward their goals. Fig. 11 illustrates the navigation of both RLoPlanner (image) and A* with ego-planner methods using RGB-D as a sensor in Scene-I with an obstacle density of 60%. It can be seen that the UAV can escape the local dilemma after

TABLE II
FLIGHT DISTANCE AND COMPUTATION TIME OF NAVIGATION ALGORITHMS IN
IRREGULAR OBSTACLE SCENE-III

	0.3 obstacle density		0.6 obstacle density	
method	$d_{fly}(m)$	$t_{com}(s)$	$d_{fly}(m)$	$t_{com}(s)$
RLoPlanner(rangefinder)	53.71	4.69	58.74	8.94
RLoPlanner(image)	54.29	5.37	59.27	9.63

RLoPlanner(image) relies on the guidance of the local target point. In contrast, UAVs using A* with ego-planner remain stuck in dead ends, emphasizing RLoPlanner's advantage in high-density obstacle environments.

G. Real-Time Performance of RLoPlanner

The real-time performance of an algorithm is a crucial performance metric. We examined the real-time performance of the proposed algorithm in Scene-III, considering obstacle densities of 30% and 0.6%. We showcased the flight distance and computation time required to complete the same flight task, and the results are presented in Table II. Given the specified hardware conditions, when conducting online control of the UAV for navigation in a scenario with lower obstacle density, both algorithms with different sensor inputs maintained a total computation time within 5.5ms. Even as the obstacle density increased to 60%, the algorithm's computation time remained below 10ms, fulfilling the requirements for real-time UAV navigation.

VI. CONCLUSION

This paper proposed RLoPlanner, a navigation framework combining recurrent maximum entropy policy gradient and traditional planner-based navigation in unknown environments with obstacles. RLoPlanner contains two advantages to make it perform better, learning complex local targets predictions from high-dimensional observations through RL, and performing precise low-level control execution through motion planners. Through extensive experiments, we demonstrated that RLoPlanner outperforms end-to-end DRL-based methods and classical planning methods in scenarios where obstacle density exceeds 50% and involves unseen objects. Specifically, our evaluation metrics showed that the proposed algorithm achieves 97.5% navigation success rate and 1.5% collision rate in 250 average experiments.

There are several issues worth investigating further. For example, the experimental environment in this work was static. In the future, we plan to further explore the hierarchical UAV navigation framework in dynamic environments, utilizing RL to predict local targets for avoiding dynamic obstacles. Additionally, considering observation and control errors is also an interesting aspect to consider.

REFERENCES

- [1] A. Gharaibeh et al., "Smart cities: A survey on data management, security, and enabling technologies," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2456–2501, Fourthquarter 2017.
- [2] M. Mohanan and A. Salgoankar, "A survey of robotic motion planning in dynamic environments," *Robot. Auton. Syst.*, vol. 100, pp. 171–185, 2018.
- [3] M. Liaq and Y. Byun, "Autonomous UAV navigation using reinforcement learning," *Int. J. Mach. Learn. Comput.*, vol. 9, no. 6, pp. 756–761, 2019.
- [4] H. Oleynikova, D. Honegger, and M. Pollefeys, "Reactive avoidance using embedded stereo vision for MAV flight," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2015, pp. 50–56.
- [5] D. Falanga, K. Kleber, and D. Scaramuzza, "Dynamic obstacle avoidance for quadrotors with event cameras," *Sci. Robot.*, vol. 5, no. 40, 2020, Art. no. eaaz9712.
- [6] B. Lindqvist, S. S. Mansouri, J. Haluška, and G. Nikolakopoulos, "Reactive navigation of an unmanned aerial vehicle with perception-based obstacle avoidance constraints," *IEEE Trans. Control Syst. Technol.*, vol. 30, no. 5, pp. 1847–1862, Sep. 2022.
- [7] S. Thrun, "Probabilistic robotics," *Commun. ACM*, vol. 45, no. 3, pp. 52–57, 2002.
- [8] J. Fuentes-Pacheco, J. Ruiz-Ascencio, and J. M. Rendón-Mancha, "Visual simultaneous localization and mapping: A survey," *Artif. Intell. Rev.*, vol. 43, pp. 55–81, 2015.
- [9] C. Richter, A. Bry, and N. Roy, "Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments," in *Proc. Robot. Res.: 16th Int. Symp. ISRR*, 2016, pp. 649–666.
- [10] H. Oleynikova, M. Burri, Z. Taylor, J. Nieto, R. Siegwart, and E. Galceran, "Continuous-time trajectory optimization for online UAV replanning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2016, pp. 5332–5339.
- [11] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2009, pp. 489–494.
- [12] W. Ding, W. Gao, K. Wang, and S. Shen, "An efficient b-spline-based kinodynamic replanning framework for quadrotors," *IEEE Trans. Robot.*, vol. 35, no. 6, pp. 1287–1306, Dec. 2019.
- [13] B. Zhou, J. Pan, F. Gao, and S. Shen, "RAPTOR: Robust and perception-aware trajectory replanning for quadrotor fast flight," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1992–2009, Dec. 2021.
- [14] H. Oleynikova, Z. Taylor, R. Siegwart, and J. Nieto, "Safe local exploration for replanning in cluttered unknown environments for microaerial vehicles," *IEEE Robot. Automat. Lett.*, vol. 3, no. 3, pp. 1474–1481, Jul. 2018.
- [15] H. X. Pham, H. M. La, D. Feil-Seifer, and L. V. Nguyen, "Reinforcement learning for autonomous UAV navigation using function approximation," in *Proc. IEEE Int. Symp. Saf., Secur., Rescue Robot.*, 2018, pp. 1–6.
- [16] C. Wang, J. Wang, Y. Shen, and X. Zhang, "Autonomous navigation of UAVs in large-scale complex environments: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2124–2136, Mar. 2019.
- [17] M. Samir, C. Assi, S. Sharafeddine, D. Ebrahimi, and A. Ghayeb, "Age of Information aware trajectory planning of UAVs in intelligent transportation systems: A deep learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 12382–12395, Nov. 2020.
- [18] I. Osband, B. V. Roy, D. J. Russo, and Z. Wen, "Deep exploration via randomized value functions," *J. Mach. Learn. Res.*, vol. 20, pp. 1–62, 2019.
- [19] A. Faust et al., "PRM-RL: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 5113–5120.
- [20] W. Gao, D. Hsu, W. S. Lee, S. Shen, and K. Subramanian, "Intention-net: Integrating planning and deep learning for goal-directed autonomous navigation," in *Proc. Conf. Robot Learn.*, 2017, pp. 185–194.
- [21] F. Xia, C. Li, R. Martín-Martín, O. Litany, A. Toshev, and S. Savarese, "ReLMoGen: Integrating motion generation in reinforcement learning for mobile manipulation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 4583–4590.
- [22] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, "Reinforcement learning with deep energy-based policies," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1352–1361.
- [23] T. Haarnoja et al., "Soft actor-critic algorithms and applications," 2018, *arxiv:1812.05905*.
- [24] W. Shi, S. Song, and C. Wu, "Soft policy gradient method for maximum entropy deep reinforcement learning," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 3425–3431.
- [25] M. Hausknecht and P. Stone, "Deep recurrent q-learning for partially observable MDPs," in *Proc. AAAI Fall Symp. Ser.*, Sep. 2015, pp. 29–37.
- [26] N. Heess, J. J. Hunt, T. P. Lillicrap, and D. Silver, "Memory-based control with recurrent neural networks," 2015, *arXiv:1512.04455*.
- [27] Y. Xue and W. Chen, "A UAV navigation approach based on deep reinforcement learning in large cluttered 3D environments," *IEEE Trans. Veh. Technol.*, vol. 72, no. 3, pp. 3001–3014, Mar. 2023.

- [28] D. Yarats, A. Zhang, I. Kostrikov, B. Amos, J. Pineau, and R. Fergus, "Improving sample efficiency in model-free reinforcement learning from images," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 12, 2021, pp. 10674–10681.
- [29] A. P. Kalidas, C. J. Joshua, A. Q. Md, S. Basheer, S. Mohan, and S. Sakri, "Deep reinforcement learning for vision-based navigation of UAVs in avoiding stationary and mobile obstacles," *Drones*, vol. 7, no. 4, 2023, Art. no. 245.
- [30] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.
- [31] N. Heess, G. Wayne, D. Silver, T. Lillicrap, T. Erez, and Y. Tassa, "Learning continuous control policies by stochastic value gradients," in *Proc. 28th Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 2944–2952.
- [32] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [33] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 478–485, Apr. 2021.



Yuntao Xue received the B.Sc. degree from the Taiyuan University of Technology, Taiyuan, China, in 2017. He is currently working toward the Ph.D. degree with the School of Aerospace Science and Technology, Xidian University, Xi'an, China. His research interests include autonomous UAV navigation and deep reinforcement learning.



Weisheng Chen received the B.S. degree in mathematics and applied mathematics from the Department of Mathematics, Qufu Normal University, Qufu, China, in 2000, the M.Sc. degree in operational research and cybernetics, and the Ph.D. degree in applied mathematics from the Department of Applied Mathematics, Xidian University, Xi'an, China, in 2004 and 2007, respectively. From 2008 to 2009, he was a Visiting Scholar with the Automation School, Southeast University, Nanjing, China. He is currently a Professor with the School of Aerospace Science and Technology, Xidian University. His research interests include multiagent systems, adaptive control, event-triggered control, distributed optimization, learning, and control.